Episodio 6 de 14: Simular 4 procesadores con un solo PC

Serie «Leer sin que te quiten el libro»: la réplica del paper «A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting» (Euro-Par 2024), explicada para principiantes.

- **Episodio anterior:** episodio 5, «Cómo se hizo la réplica, y cuánto fiarse».
- **Episodio siguiente:** episodio 7, «Rendimiento: quién gana, y el misterio de Folly».

Lo esencial de este episodio:

- Una máquina virtual con 4 nodos NUMA sería un decorado. Se usaron distancias reales: 35 ns dentro de un clúster, 47 entre clústeres y 86 entre núcleos P.
- Separar 4 hilos cuesta mucho donde hay contención (Autores pierde entre un 32 % y un 54 %) y nada donde no la hay (±2 % con un millón de pilas).
- Lo que importa es compartir la caché L2, no los nanosegundos. Y el castigo aquí es mucho más suave que entre sockets reales: 1,35× frente a unas 3×.

Parte 5. Simular 4 Xeon con una sola CPU

5.1 Qué significa de verdad «simular 4 sockets»

Una máquina de 4 sockets es distinta por dos motivos:

1. **Distancia entre núcleos:** pasar una línea de caché a otro chip es varias veces más lento.
2. **Distancia a la memoria (NUMA):** cada chip tiene su RAM; la del vecino está más lejos.

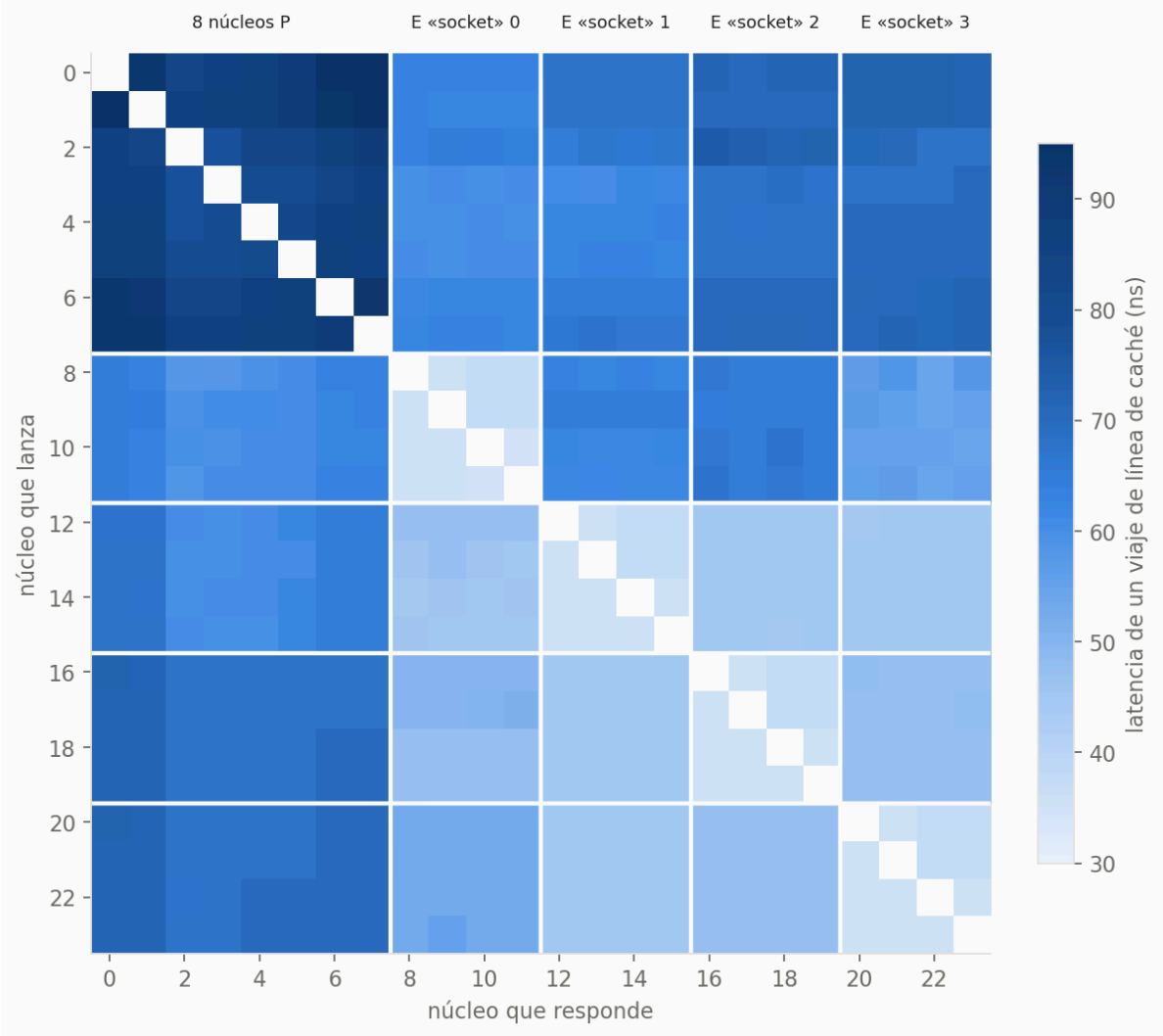
Una máquina virtual con 4 nodos NUMA engaña al sistema operativo: dice que hay 4 nodos, pero **ningún viaje tarda más**. Sería un decorado. Por eso se descartó.

Lo que importa aquí es lo primero: la línea caliente que viaja. Y eso sí se puede imitar con hardware real, si esta CPU tiene **distancias reales** que aprovechar.

5.2 Medir antes de suponer

El Core Ultra 9 285 tiene 8 núcleos rápidos (**P**) y 16 eficientes (**E**). Los E están en **4 clústeres de 4 núcleos**, y cada clúster comparte una caché L2. Exactamente 4 grupos: suena a 4 sockets en miniatura. Pero hay que comprobarlo.

`herramientas/latencia_nucleos.cpp` lanza una «pelota» entre cada par de núcleos y mide cuánto tarda:



Latencia entre cada par de núcleos

Cómo leerla: cada cuadrito es un par de núcleos; **más oscuro es más lento**. Los cuatro bloques claros de la diagonal son los clústeres: dentro de cada uno, los viajes son muy rápidos.

Viaje entre…	Mediana	Respecto a «mismo clúster»
Núcleos E del mismo clúster	**35 ns**	1×
Núcleos E de clústeres distintos	**47 ns**	1,35×
Núcleo P y núcleo E	67 ns	1,9×
Dos núcleos P	**86 ns**	2,5×

Dos cosas importantes:

- **Los clústeres sí se comportan como sockets:** cerca dentro, más lejos fuera. La hipótesis se confirma.
- **Sorpresa:** los núcleos P son los **más lejanos entre sí**. Son los más rápidos calculando y los más lentos pasándose datos. No lo habríamos adivinado.

5.3 ¿Cuánto se parece a los Xeon reales?

En procesadores de esa generación, una transferencia dentro del socket ronda los 40-45 ns, y entre sockets los 130-140 ns: más o menos **3×**. Ese dato es indirecto, porque viene de

máquinas parecidas y no de la del paper.

Aquí, entre clústeres, la penalización es **1,35×**. Es decir: la miniatura tiene la forma correcta, pero el castigo por distancia es mucho más suave. Va en la buena dirección, pero no llega a la misma fuerza. Hay que tenerlo presente.

5.4 El diseño de la batería de afinidad

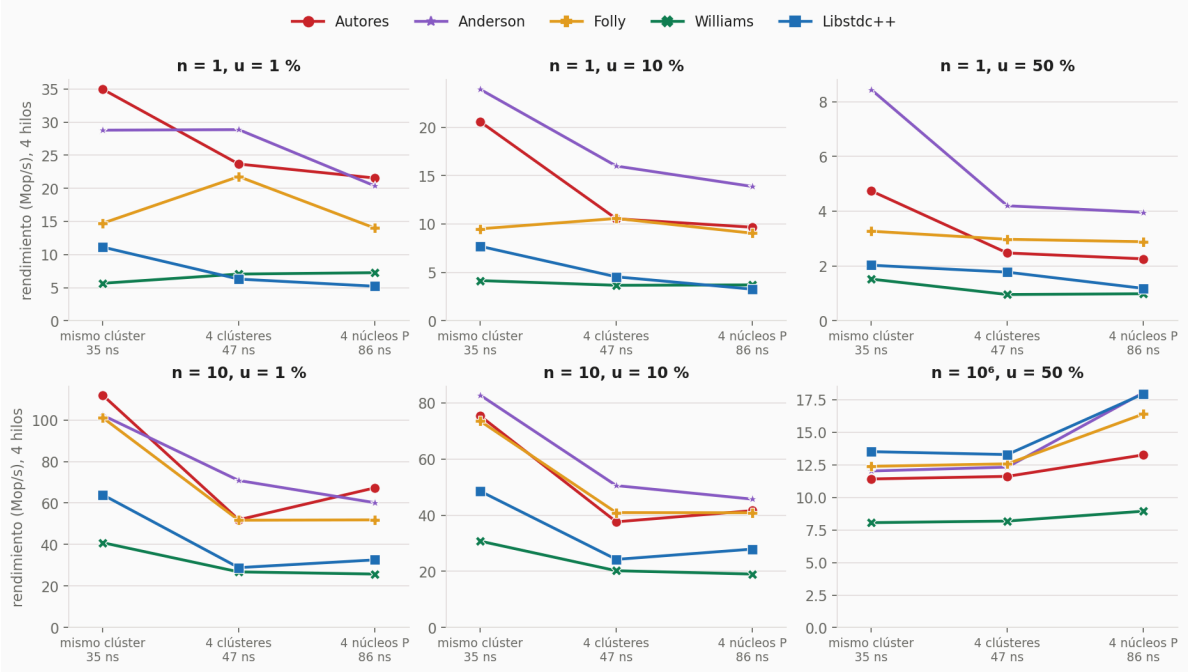
Afinidad significa decidir en qué núcleo exacto corre cada hilo. Se hace con `herramientas/fijar_hilos.c`, una pequeña librería que se carga por delante del programa e intercepta la creación de hilos. **El binario oficial no se modifica**: solo cambia dónde cae cada hilo. Se comprobó leyendo el sistema: cada hilo estaba en su núcleo.

Colocación	Núcleos	Qué imita
`E…-compacto`	llena un clúster, luego el siguiente	todos los hilos en el mismo socket, mientras quepan
`E…-disperso`	reparte entre los 4 clústeres	hilos repartidos por los 4 sockets
`P…`	núcleos P	caso de máxima distancia (86 ns), con núcleos más potentes
`todos24`	los 24	toda la máquina, con los hilos fijados
`sistema`	sin fijar	lo que hizo el paper: lo decide Linux

El experimento más limpio es **4 hilos**: mismo número de hilos, y en `compacto` frente a `disperso` también el mismo tipo de núcleo. **Lo único que cambia es la distancia.** Es un experimento de **dosis-respuesta**: subimos la dosis (35 → 47 → 86 ns) y miramos cuánto cae cada algoritmo.

6.4 La simulación de los 4 Xeon: ¿importa la distancia?

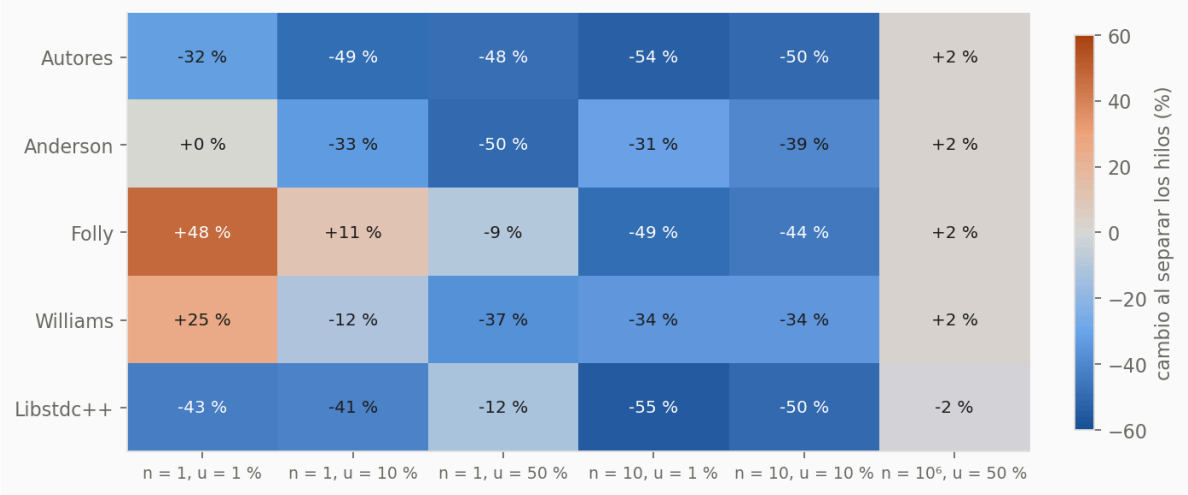
Vuelve a la Parte 5 si no recuerdas el montaje. La pregunta es simple: **con los mismos 4 hilos, ¿qué pasa si los alejamos?**



Rendimiento con 4 hilos a distancia creciente

Cómo leerla. Tres posiciones, de izquierda a derecha: los 4 hilos en un mismo clúster (35 ns por viaje), uno en cada clúster (47 ns) y en 4 núcleos P (86 ns). Si la distancia castiga, las líneas bajan. El tercer punto no es del todo limpio, porque los núcleos P son además más potentes; el salto más honesto es el del primer al segundo punto.

El mapa resume ese primer salto, usando solo núcleos E, para que lo único que cambie sea la distancia:



Cambio de rendimiento al separar 4 hilos

Azul significa que pierde al separar, y naranja, que gana.

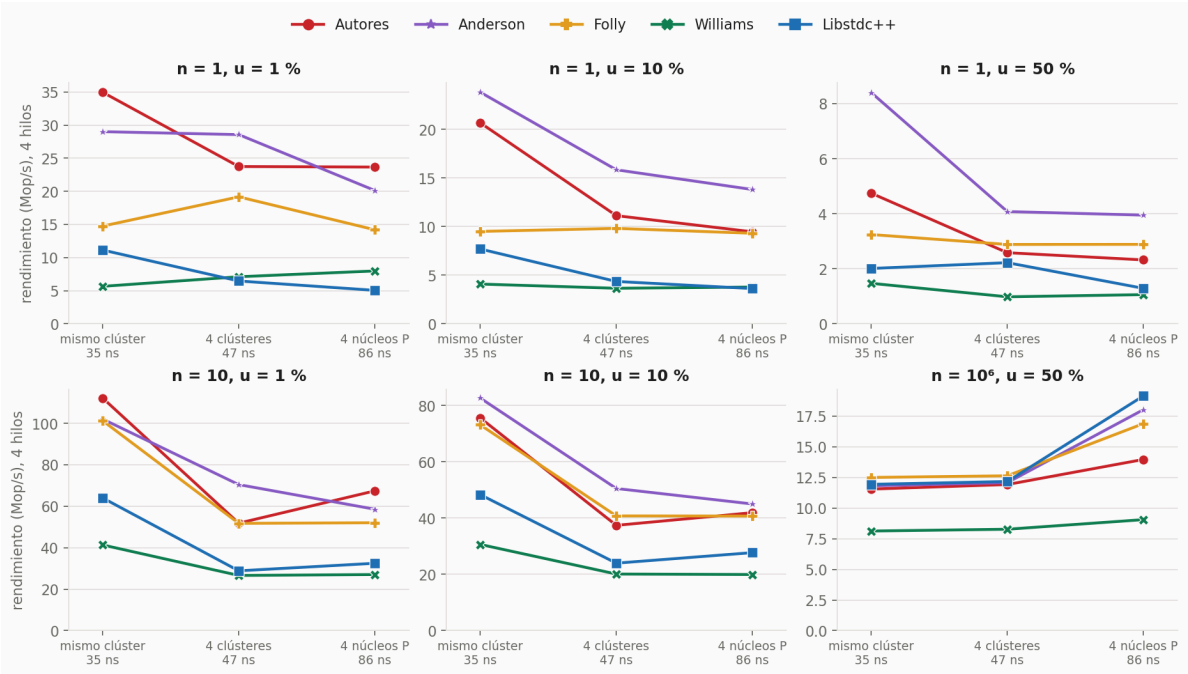
Lo que salió:

1. **Separar cuesta mucho donde hay contención.** Autores pierde entre un 32 % y un 54 %; Libstdc++, entre un 12 % y un 55 %. Con solo 4 hilos.
2. **Donde no hay contención, no cuesta nada.** En n = 10^6 todo se mueve un ±2 % (sin Docker, hasta un +3 %). Ese es el control del experimento: si el castigo apareciera

también aquí, sospecharíamos del montaje. Sin cuaderno compartido, la distancia da igual.

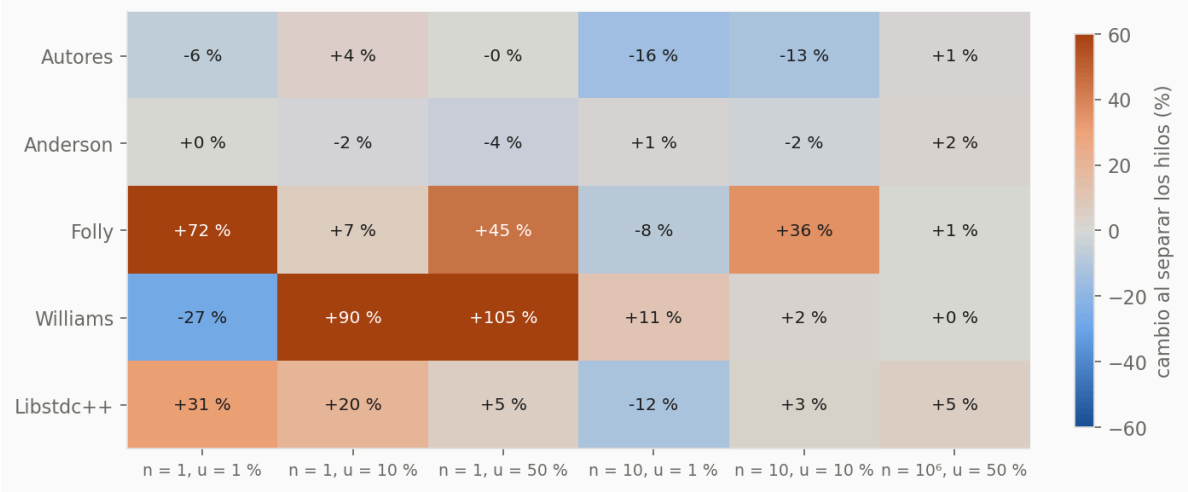
3. **El castigo es mayor que la distancia.** El viaje es 1,35 veces más largo, pero se pierde hasta la mitad del rendimiento. Probablemente porque, dentro de un clúster, la línea caliente ni siquiera sale de la L2 compartida. No es una regla de tres.
4. **Anderson aguanta mejor con pocas escrituras:** +0 % con n = 1 y u = 1 %, frente al −32 % de Autores.
5. **Hay excepciones que van al revés.** Folly gana un 48 % y Williams un 25 % al separar, con n = 1 y u = 1 %. Los dos dependen de bucles de `CAS`. La sospecha es que, con los hilos más lejos, chocan menos entre sí, parecido a la «pausa» de la que habla el paper. No está demostrado.

¿Y sin Docker? Con los mismos programas oficiales, fuera de Docker, sale casi calcado. Autores vuelve a perder entre un 32 % y un 54 %; Anderson, un 2 % con n = 1 y u = 1 %; y las dos excepciones se repiten: Folly gana un 30 % y Williams un 26 %. Las cifras están en la tabla «Efecto de separar los hilos entre «sockets» (nativo, binarios oficiales)» de `tablas.md`.



Rendimiento con 4 hilos a distancia creciente, sin Docker y con los programas oficiales

El efecto se apaga con 8 hilos o más. Con 8 hilos, incluso la colocación compacta ocupa ya 2 clústeres: el cuaderno ya tiene que salir de la sala, y que vaya a dos o a cuatro salas importa mucho menos.



Cambio de rendimiento al separar 8 hilos

Ojo con una cifra de `tablas.md` que llama la atención: sin Docker, con 8 hilos, n = 1 y u = 10 %, Folly «gana» un 261 % al separar. No es que separar le ayude tanto. Es que en la colocación compacta se hunde: sus 3 rondas dan entre 0,35 y 1,33 Mop/s, cuando con 4 hilos compactos da 9,5. Con Docker ya hubo una ronda así (0,28). No se investigó.

¿Y la rareza de Williams? La batería mostró algo extraño. Con 2 hilos, n = 1 y u = 50 %, Williams daba 3,05 Mop/s en un mismo clúster y 0,04 en clústeres distintos. Una caída tan brutal hacía sospechar de un par de núcleos defectuoso, así que se probaron 14 pares de 4 clases, con 3 rondas cada uno; después, sin Docker y con el programa oficial, con 10.

Programa oficial, en Docker (Mop/s, mediana de cada clase):

Dónde están los 2 hilos	Pares	Autores	Anderson	Folly	Williams	Libstdc++
Mismo clúster	4	6,21	11,82	6,75	3,05	5,07
Clústeres distintos	7	3,41	6,58	4,68	**0,09**	3,10
P ↔ P	2	3,29	6,41	4,38	1,65	3,55
P ↔ E	1	3,41	6,52	4,40	1,65	3,03

`bench.cpp` del equipo, sin Docker:

Dónde están los 2 hilos	Pares	Autores	Libstdc++	Williams	Mutex
Mismo clúster	4	8,83	4,33	2,02	3,91
Clústeres distintos	7	4,41	2,58	1,16	3,91
P ↔ P	2	4,23	3,02	1,01	4,03
P ↔ E	1	4,34	2,70	1,09	4,00

Programa oficial, sin Docker, con 10 rondas por par en vez de 3 (calculado de `nativo_oficial/rendimiento.csv` igual que la primera tabla; los valores par a par están en `tablas.md`):

Dónde están los 2 hilos	Pares	Autores	Anderson	Folly	Williams	Libstdc++
Mismo clúster	4	6,21	11,61	6,69	2,93	5,03
Clústeres distintos	7	3,45	6,44	4,70	**0,70**	2,97
P ↔ P	2	3,32	6,47	4,46	1,44	3,60
P ↔ E	1	3,39	6,68	4,47	1,57	3,21

Estas tablas dejan tres lecciones:

1. **Lo que importa es compartir L2, no los nanosegundos.** Según el mapa de latencias, ir entre clústeres cuesta 47 ns y entre núcleos P, 86 ns. Pero Autores rinde casi lo mismo en los dos casos (3,41 y 3,29). El salto grande está antes, al dejar de compartir caché: −45 % con el programa oficial en Docker, −44 % con el mismo programa sin Docker y −50 % con `bench.cpp`. No es una rampa, es un **escalón**. Es como trabajar en la misma mesa: en cuanto tienes que levantarte, da casi igual si vas a la sala de al lado o a la del fondo.
2. **Al Mutex esta prueba le da igual** (3,91 en ambos casos). Una explicación posible es que los hilos que esperan duermen, así que el cuaderno viaja mucho menos. Se habla de esto en la Parte 6.6.
3. **Lo de Williams no es un par defectuoso.** Pasa en los 7 pares de clústeres distintos, con caídas del 92 % al 99 % según el par. Además es inestable: va de 0,02 a 1,09 Mop/s, mientras que en un mismo clúster se queda clavado entre 3,03 y 3,06. Sin Docker, con el mismo programa, también se hunde en los 7 pares, pero bastante menos: entre un 59 % y un 79 % según el par, un 76 % si se toman todas las rondas juntas. Y sigue siendo inestable: sus 70 rondas entre clústeres van de 0,34 a 1,79 Mop/s, mientras que en un mismo clúster vuelve a quedarse clavado, entre 2,92 y 2,98.

Entonces, ¿quién tiene la culpa? Se investigó como un detective, descartando sospechosos:

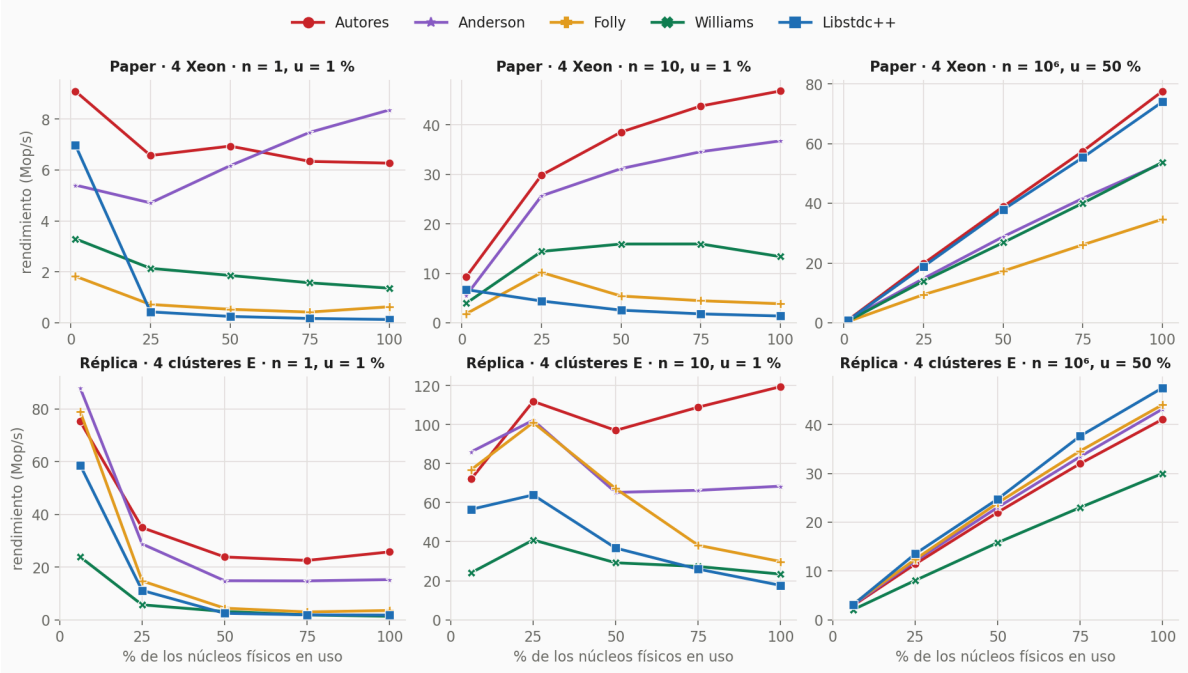
Sospechoso	Prueba	Resultado
Un par de núcleos defectuoso	7 pares distintos	Pasa en los 7: descartado
Docker	`bench.cpp` **dentro** de Docker, en los mismos pares	1,12 a 1,19 Mop/s, sin hundimiento: descartado
Docker, segunda prueba	El programa oficial **fuera** de Docker, 10 rondas por par	Se sigue hundiendo (0,70 frente a 2,93 Mop/s): Docker no lo provoca. Pero se hunde menos que dentro (0,09)
`boost::atomic` (el artefacto lo usa en lugar de `std::atomic`)	`bench.cpp` con la cabecera del artefacto, en Docker	1,15 a 1,28 Mop/s, sin hundimiento: descartado
Algo del programa oficial	Solo pasa con `bench_stack`, dentro y fuera de Docker; con `bench.cpp`, en ninguno de los dos	**Sigue abierto**, y es el sospechoso principal

Queda una pregunta nueva: ¿por qué dentro de Docker se hunde más? No se puede culpar a Docker sin más, porque dentro y fuera cambian más cosas: Boost (1.74 frente a 1.83), el compilador (12.3 frente a 12.4) y glibc (2.35 frente a 2.39). Como el Williams del artefacto usa

`boost::atomic`, el cambio de versión de Boost es un candidato natural. Es solo una sospecha: no se midió.

El efecto solo aparece en `bench_stack`. Su bucle es más apretado: no hay un generador de azar lento entre operación y operación, y usa `compare_exchange_weak`. La sospecha es que dos hilos en clústeres distintos se quedan pisándose por turnos, y que el trabajo extra de `bench.cpp` rompe ese ritmo, parecido a la «pausa» de la que habla el paper. Pero eso **no está demostrado**.

La miniatura frente al original. Última prueba: dibujar el paper y la réplica según el porcentaje de máquina ocupada.



Paper y réplica según el porcentaje de núcleos en uso

- **n = 10, u = 1 %: se parece.** Autores encabeza en ambos a partir del 25 % de la máquina.
- **n = 10⁶, u = 50 %: la forma se parece.** Rectas que suben en los dos, aunque el orden entre las de arriba cambia.
- **n = 1, u = 1 %: no se parece.** En el paper, Anderson sube y adelanta a Autores. Aquí se queda plano.

Entonces, ¿se simularon los 4 Xeon? Solo en parte, y conviene decirlo claro. Es como un simulador de vuelo de juguete: enseña de verdad qué pasa al alejar los hilos, y esa física es real, medida con hardware real. Pero el castigo es mucho más suave (1,35× frente a unas 3×), no hay memoria repartida entre chips y hay 16 núcleos E, no 72. Sirve para ver **qué algoritmos son sensibles a la distancia**. No sirve para predecir los números de la máquina del paper.